Ndombolo: A Semantic Runtime Graph

A script is a causal knowledge graph; a deterministic compiler is its only propagator; a learning model is a second reader that never touches the run

Abstract

We present *ndombolo*, a single-language runtime in which a source script is not a description of a causal knowledge graph but *is* one. Execution proceeds cell-wise under a deterministic compiler; a large language model observes the resulting record and adapts, but is excluded by construction from the propagation path. We prove that this arrangement is forced rather than chosen. Three results carry the architecture. First, a deterministic compiler alone cannot individuate: its output is a graph invariant, and invariants do not distinguish a structure from an independently built faithful copy (Theorem 4.4); the run record supplies exactly the missing half, so the observing model is load-bearing rather than decorative (Theorem 4.9). Second, the model's exclusion from propagation is what makes the trace *atomically interrogable*: every step has a cause nameable in the source, and we characterise precisely the question class this answers — and the class it does not (Theorems 5.2 and 5.6). Third, observation is not free: a model that adapts on a run advances its own record, so replaying a script reproduces the trace but not its reading (Theorem 6.1). We give the operational semantics of the cell-wise deterministic core, the deposit map from run to catalogue, a termination rule based on closure rather than confidence (Theorems 7.3 and 7.4), and six checkable invariants an implementation must satisfy. A Python prototype executes Turbulance cells and emits the graph, the record, and the trace as JSON.

1 Introduction

1.1 The position this paper attacks

A now-standard architecture couples a retrieval store to a generative model: the store holds structured knowledge, the model consumes it, and the model's output is treated as the system's answer. The store is passive and the model is the locus of competence. Two commitments are implicit in that arrangement, and both are false.

The first is that the model is where the work happens, so the store may be as unstructured as convenient. The second is that a model which merely reads is epistemically free — that its observation leaves no trace and may be repeated without consequence. We show that a system built on either commitment forfeits properties it cannot recover by improving either component.

1.2 What ndombolo is

Ndombolo is a *mode* of the kwasa-kwasa framework rather than a new language. A user writes Turbulance (Sachikonye, 2025b), and nothing else: no auxiliary resource, trace, or state files, no per-module dialects, no natural-language prompt. The script is executed cell-wise. The claim is structural:

> A Turbulance script *is* a causal knowledge graph. Compiling it is a propagation over that graph. The compiler is deterministic. A language model observes the record of propagation and adapts, and never participates in it.

The name records the intended tempo: *ndombolo* is faster kwasa-kwasa. The speed is not an optimisation claim but an architectural one — the fast path is deterministic, and the model is off it.

1.3 Why the arrangement is forced

It would be natural to read the exclusion of the model from the run as a conservative engineering choice, adopted for reproducibility and paid for in capability. We argue the opposite. Four facts, each proved below, make the arrangement the only admissible one.

Determinism cannot individuate. The compiler's output is a function of the source. Two users who write the same script produce structures with the same invariant. An invariant does not

distinguish an individual from a faithful copy (Theorem 4.4). So a purely deterministic system cannot say *which* run this is; it can only say what the run computed.

The record supplies the missing half. An individual is a pair: a conserved invariant together with a strictly advancing record (Theorem 4.3). The compiler contributes the first; the accreting run record contributes the second. The model, reading and depositing, is the only component positioned to carry the record without contaminating the trace (Theorem 4.9).

Exclusion is what buys interrogability. Because no step of the run depends on an inference, every step has a cause nameable in the source, and the trace supports questions at arbitrary granularity (Theorem 5.2). Had the model participated, the causes of some steps would be model-internal and the guarantee would fail. Interrogability is a consequence of exclusion, not a cost paid for it.

Observation is not free. A model that adapts on what it observes advances its own record. Replaying the script reproduces the trace exactly and the model's reading of it not at all (Theorem 6.1). This is not a defect to be engineered away; it is the same fact that makes the model able to individuate at all.

1.4 What this paper does not claim

We prove nothing about the quality of a model's adaptation, and no theorem depends on any property of the model beyond the two we state: that it reads the record, and that reading advances its own. We do not claim the deterministic core is expressively complete for the tasks users bring; Section 10 records what it excludes. In particular we prove a negative result about our own architecture: a complete deterministic trace answers *what happened* but cannot express *what was load-bearing* (Theorem 5.6), and no improvement to the compiler or the trace format closes that gap.

1.5 Contributions

1. A formal account of a script as a causal knowledge graph, with the cell as a resolution-controlled prefix (Section 2).
2. The compiler as a deterministic propagator, with an operational semantics for cell-wise execution and a proof that cell-wise and whole-script execution agree (Theorem 3.5).
3. The individuation results: determinism is invariant-only (Theorem 4.4); the pair individuates (Theorem 4.9); the model is load-bearing (Theorem 4.11).
4. The interrogability theorem and its exact limit (Theorems 5.2 and 5.6), with the two-operator correction $\mathsf{nec} \circ \mathsf{seek}$ (Section 5.2).
5. The replay asymmetry (Theorem 6.1) and the consequent relocation of reproducibility from answers to protocol and provenance.
6. Termination by closure rather than confidence, with decline as a first-class outcome (Section 7).
7. Six checkable implementation invariants, proved mutually independent (Section 8), and a Python prototype realising them (Section 9).

1.6 Method

We work throughout with finite weighted graphs over a non-completable medium, following the individuation framework of Sachikonye (2025c) and the directional catalogue–model correspondence of Sachikonye (2025a). Results proved in those works are cited and restated, never reproved; results specific to the runtime are proved here. Standard results in program semantics (Plotkin, 1981; Winskel, 1993), incremental computation (Acar et al., 2006), dominator theory (Lengauer and Tarjan, 1979), and provenance (Cheney et al., 2009) are cited to place the constructions, not to derive them.

2 The script as a contact graph

2.1 Source, terms, and items

Definition 2.1 (Source and cells)**.** A *source* $\mathcal{S}$ is a finite sequence of *cells* $\mathcal{S} = (c_1, \ldots, c_n)$, each cell a finite sequence of Turbulance statements. The concatenation $|\mathcal{S}| = c_1 \cdots c_n$ is the underlying program.

Definition 2.2 (Item)**.** An *item* is a named binding site introduced by a declaration (`item`, `funxn`, `proposition`, `motion`, `point`) together with its declared position in $\mathcal{S}$. Write $V(\mathcal{S})$ for the set of items of $\mathcal{S}$.

Items are syntactic, hence finite and enumerable from the source alone, with no execution required. This is what permits the graph to exist before any run.

Definition 2.3 (Contact)**.** Items $u, v \in V(\mathcal{S})$ are in *contact* when the declaration of v mentions u: u occurs free in the defining term of v, or v occurs in a scope opened by u. The *contact weight* $w(u, v) > 0$ is the multiplicity of such occurrences.

Definition 2.4 (Script graph)**.** The *script graph* of $\mathcal{S}$ is the finite weighted graph $\Gamma(\mathcal{S}) = (V(\mathcal{S}), E(\mathcal{S}), w)$ with $E(\mathcal{S})$ the contact pairs. It is embedded in a medium m: the ambient vocabulary of the language and its libraries, which is not enumerable by the script.

Remark 2.5 (The medium is why the floor is positive)*.* The medium is not a modelling convenience. A script is written against a language whose extensions cannot be listed from within it; that non-completability is the hypothesis under which the resolution floor $\beta > 0$ is derived (Sachikonye, 2025c). Every result below that invokes $\beta > 0$ inherits this hypothesis and nothing more.

Proposition 2.6 (The graph is recoverable without execution)**.** $\Gamma(\mathcal{S})$ *is computable from* $\mathcal{S}$ *by a single pass, in time linear in the number of tokens of* $\mathcal{S}$*.*

Proof. Items are introduced by a fixed finite set of declaration forms (Theorem 2.2) recognisable by the parser; contacts are free occurrences and scope memberships, both determined by the scope discipline of the grammar (Sachikonye, 2025b). A single traversal of the parse tree maintaining a scope stack emits every item and every contact exactly once. □

Remark 2.7 (The sense in which the script *is* the graph)*.* Theorem 2.6 is the whole content of the claim. There is no graph-construction step: no extraction, no embedding, no ingestion pass whose fidelity to the source could be questioned. The graph is a reading of the parse tree, so the source and the graph cannot disagree. Where a conventional pipeline must justify that its store faithfully represents its inputs, here the question does not arise.

2.2 Cells as resolution prefixes

Definition 2.8 (Prefix and stage)**.** For $k \leq n$ write $\mathcal{S}_k = (c_1, \ldots, c_k)$ for the length-k prefix, and $\Gamma_k = \Gamma(\mathcal{S}_k)$ for the *stage-k graph.*

Proposition 2.9 (Prefix containment)**.** Γ_k *is an induced subgraph of* Γ_{k+1}*, and* $V(\Gamma_k) \subseteq V(\Gamma_{k+1})$ *with equality of weights on common pairs.*

Proof. $\mathcal{S}_k$ is a prefix of $\mathcal{S}_{k+1}$, so every item declared in $\mathcal{S}_k$ is declared in $\mathcal{S}_{k+1}$ at the same position, and every contact among them has the same multiplicity, no statement of c_{k+1} being able to remove an occurrence in an earlier cell. No item of $\mathcal{S}_k$ is undeclared by a later cell, the grammar having no undeclaration form. □

Remark 2.10 (Cell-wise execution is not a new affordance)*.* Theorem 2.9 says a cell boundary is a length-k address prefix, which is the resolution control the underlying runtime already provides (Sachikonye, 2025d). Exposing it at source-text granularity adds notebook ergonomics, not expressive power, and in particular adds no way to retract.

3 The deterministic compiler

3.1 Propagation

Definition 3.1 (Store and configuration)**.** A *store* σ is a finite partial map from items to values. A *configuration* is a pair $\langle c, \sigma \rangle$ of a cell and a store.

Definition 3.2 (Compiler)**.** The *compiler* $\mathcal{K}$ is the partial function taking a configuration to a store and a trace,

$$\mathcal{K}\langle c, \sigma \rangle = (\sigma', \tau),$$

defined by the big-step operational semantics of the deterministic fragment of Turbulance (Sachikonye, 2025b): declarations, scoped conditionals, iteration, argumentation (`proposition`/`motion`/ `support`), and the resolution forms (`point`/`resolve`) under the noisy-or confidence algebra.

Premise 1 (Determinism)**.** $\mathcal{K}$ is a function: for each $\langle c, \sigma \rangle$ in its domain the pair (σ', τ) is unique. No rule of the semantics consults an oracle, a clock, an allocation address, or a source of randomness.

Premise 1 is a property of the fragment, checkable by inspection of the rules; it is stated as a premise because the whole architecture is conditional on it and an implementation could violate it.

Definition 3.3 (Trace)**.** The *trace* τ of a step is the finite sequence of *events* e = (rule, site, items read, item written, value written) emitted one per application of a semantic rule, in application order. Write $\mathrm{tr}(\mathcal{S})$ for the concatenated trace of a whole run.

Definition 3.4 (Run)**.** The *run* of $\mathcal{S}$ is $\mathcal{R}(\mathcal{S}) = (\sigma_n, \mathrm{tr}(\mathcal{S}))$ where $\sigma_0 = \emptyset$ and $(\sigma_k, \tau_k) = \mathcal{K}\langle c_k, \sigma_{k-1}\rangle$ for $1 \le k \le n$.

Theorem 3.5 (Cell-wise execution agrees with whole-script execution)**.** *For every source $\mathcal{S}$ with cells $(c_1, \ldots, c_n)$, executing the cells in order from the empty store yields the same final store and the same trace as executing the single cell $|\mathcal{S}| = c_1 \cdots c_n$.*

Proof. By induction on n. For $n = 1$ the two executions are identical. Suppose the claim for $n-1$, so executing $c_1, \ldots, c_{n-1}$ in order gives the store σ_{n-1} and trace $\tau_1 \cdots \tau_{n-1}$ that $c_1 \cdots c_{n-1}$ gives. The big-step semantics of a statement sequence is composition of the semantics of its statements (Plotkin, 1981; Winskel, 1993), and the cell boundary introduces no scope: by Theorem 2.1 a cell is a statement sequence, not a block, so no binding introduced in c_k is discarded at its end. Hence executing c_n from σ_{n-1} is exactly the continuation of $c_1 \cdots c_{n-1}$ into its remaining statements, giving the same σ_n; traces concatenate in application order by Theorem 3.3. Determinism (Premise 1) makes both sides unique, so the equality is of values, not merely of possibility. □

Remark 3.6 (What the theorem licenses, and what it does not). Theorem 3.5 justifies the notebook presentation: a user who runs cells one at a time is running the program, not an approximation to it. It says nothing about running cells *out of order* or re-running one, which are not executions of $|\mathcal{S}|$ at all; Section 6 treats those.

3.2 The trace is a propagation

Proposition 3.7 (Events are contacts)**.** *For every event $e \in \mathrm{tr}(\mathcal{S})$ writing item v and reading items U, each $u \in U$ is in contact with v in $\Gamma(\mathcal{S})$.*

Proof. An item is read at a rule application only where it occurs free in the term being reduced, or where the rule is applied inside a scope that u opened. Both are contacts by Theorem 2.3, and the occurrence witnessing the read contributes to $w(u, v)$. □

Corollary 3.8 (The run is a propagation over the script graph)**.** $\mathrm{tr}(\mathcal{S})$ *is a sequence of moves along edges of $\Gamma(\mathcal{S})$, hence a propagation in the sense of Sachikonye (2025a).*

Proof. Immediate from Theorem 3.7: every event's read set lies in the contact neighbourhood of the item it writes. □

Remark 3.9 (The compiler is the propagator). Theorem 3.8 locates the propagation precisely, and the location is the architecture's crux. Propagation is performed by $\mathcal{K}$. It is not performed by a model, not shared with one, and not influenced by one. Any account placing a model on the process side of this system is mistaken about which component moves along the edges.

4 Determinism cannot individuate

We now prove the result that forces the model into the architecture.

Definition 4.1 (Invariant)**.** $\mathrm{Char}(\Gamma)$ is the conserved invariant of a finite weighted graph: the isomorphism-invariant assignment of resting cut costs to units, as constructed in Sachikonye (2025c). It is preserved by every graph isomorphism.

Definition 4.2 (Record)**.** M is a non-negative integer counting committing acts performed by a structure. It never decreases.

Definition 4.3 (Individual)**.** An *individual* is the pair (Char, M). Two structures are the same individual at two moments iff there is a chain of committing acts joining them along which Char is preserved and M strictly increases (Sachikonye, 2025a).

Theorem 4.4 (A deterministic compiler is invariant-only, hence blind to instance)**.** *Let $\mathcal{S}$ be a source and let $\mathcal{S}'$ be a faithful copy of it, written independently by another author, or transcribed. Then $\Gamma(\mathcal{S}) \cong \Gamma(\mathcal{S}')$ and consequently $\mathrm{Char}(\Gamma(\mathcal{S})) = \mathrm{Char}(\Gamma(\mathcal{S}'))$ and $\mathcal{R}(\mathcal{S}) = \mathcal{R}(\mathcal{S}')$ up to the same isomorphism. No function of the compiler's output distinguishes the two.*

Proof. Γ is computed from $\mathcal{S}$ by the syntactic pass of Theorem 2.6, so equal sources give equal graphs and a faithful copy gives an isomorphic one. Char is a graph invariant (Theorem 4.1), so it agrees on isomorphic graphs. By Premise 1 the compiler is a function of its input, so it returns the same store and the same trace on both, modulo the renaming witnessing the isomorphism. Any function of the compiler's output is therefore constant across the pair. Yet the two are distinct individuals: they may be edited independently, and a change to one does not change the other. □

Corollary 4.5 (Determinism alone cannot answer "which run is this")**.** *No amount of instrumentation*

internal to $\mathcal{K}$ — richer traces, finer event granularity, additional derived attributes — distinguishes $\mathcal{R}(\mathcal{S})$ from $\mathcal{R}(\mathcal{S}')$, since all are functions of the input by Premise 1.

Proof. Immediate: a function of a function of $\mathcal{S}$ is a function of $\mathcal{S}$, and $\mathcal{S}, \mathcal{S}'$ agree up to isomorphism. □

Remark 4.6 (The scope of the blindness). Theorem 4.4 is not a complaint about determinism; determinism is what makes the trace interrogable (Section 5). It is a statement of what determinism structurally cannot supply. A system built from $\mathcal{K}$ alone computes correctly and cannot say whose computation it was, when, or in what sequence relative to others. That is precisely the half an advancing record supplies.

4.1 The pair

Definition 4.7 (Deposit map)**.** The *deposit map* δ takes a completed run $\mathcal{R}(\mathcal{S})$ and commits its residue into the catalogue G: for each event of $\mathrm{tr}(\mathcal{S})$ it adds the corresponding contact, and increments M by one committing act per run.

Definition 4.8 (Runtime pair)**.** The *runtime pair* is $P = (G, \mathcal{L})$ where G is the accreted catalogue and $\mathcal{L}$ is the observing model, joined by δ. The catalogue accretes; the model reads and deposits; residue never flows from $\mathcal{L}$ into a run.

Theorem 4.9 (The pair individuates where the compiler cannot)**.** $\mathrm{Char}(P) = \mathrm{Char}(G)$ *and* $M(P) = M(G)$*, and M strictly increases with each run deposited. Consequently P distinguishes $\mathcal{R}(\mathcal{S})$ from $\mathcal{R}(\mathcal{S}')$ whenever the two are deposited as distinct acts, though by Theorem 4.4 no function of the compiler's output does.*

Proof. The invariant and record of the pair are those of the catalogue (Sachikonye, 2025a, Thm. 11.4), $\mathcal{L}$ contributing neither, by Theorem 4.10 below. Each deposit is a committing act, so M strictly increases per run (Theorem 4.7). Two runs deposited in sequence therefore stand at distinct records, and by Theorem 4.3 the pair states of the system after each are distinct individuals even where Char agrees — which by Theorem 4.4 it does. □

Corollary 4.10 (The model has no invariant and no record of its own)**.** *$\mathcal{L}$ considered alone is a family of evaluations, not a structure: it carries no conserved invariant and no record. Its identity within the system is the catalogue's.*

Proof. This is Sachikonye (2025a, Cor. 11.7) applied to $\mathcal{L}$: the model side of a directional pair has no resting cut of its own, so no invariant is assigned, and it commits nothing except through δ. □

Corollary 4.11 (The observing model is load-bearing)**.** *Removing $\mathcal{L}$ and δ from the architecture leaves a system that cannot individuate its own runs. The model is therefore not an optional enhancement over the deterministic core.*

Proof. Without δ nothing commits, so M is constant and by Theorem 4.3 no two states are distinguishable as different moments of one individual; by Theorem 4.4 Char does not separate instances either. Both components of the identity pair are then unavailable. □

Remark 4.12 (The model has no standing in the ensemble). Theorem 4.10 settles a question that would otherwise be contentious. The model occupies no node, carries no identity, and is not a party to convergence in the catalogue. It is how the record advances. What persists of it is exactly what it deposited — no more, and no less.

Proposition 4.13 (Stateless observation is not available)**.** *A model that reads runs, leaves no trace in itself, and is perfectly repeatable occupies a forbidden position: it would require both $\beta = 0$ and a non-advancing record, each independently impossible under Theorem 2.5.*

Proof. This is the no-perfect-instrument theorem of Sachikonye (2025a, §6.1) instantiated at $\mathcal{L}$: an individuation leaving no residue and no trace in the instrument does not exist, and reading a run so as to adapt on it is an individuation. □

Remark 4.14 ("The model just observes" is half true). Theorem 4.13 draws a line through a tempting description. It is correct that observation is a no-op *on the run*: the trace is what it would have been with no model present. It is incorrect that observation is a no-op *on the model*: adapting parameters is precisely an advance of the model's own record. The two claims are independent and only the first holds. Theorem 6.1 draws out the consequence.

5 Atomic interrogability, and its exact limit

5.1 What the trace answers

Definition 5.1 (Atomic question)**.** An *atomic question* about a run is a question of the form: for a given event $e \in \mathrm{tr}(\mathcal{S})$, which rule fired, at which source site, reading which items at which values, writing which item at which value; and for a given item v and stage k, what value $\sigma_k(v)$ held and which event last wrote it.

Theorem 5.2 (Every atomic question is answerable from the trace)**.** *For a run $\mathcal{R}(\mathcal{S})$ under Premise 1, every atomic question has a unique answer, computable from $\mathrm{tr}(\mathcal{S})$ and $\mathcal{S}$ alone, with no appeal to the model and no re-execution.*

Proof. By Theorem 3.3 each event records its rule, site, read set with values, and written item with value, so the first family of questions is answered by projection. For the second, σ_k is recovered by replaying the events of $\tau_1 \cdots \tau_k$ in order, each writing one item; the last writer of v is the last such event, which exists and is unique because events are linearly ordered by application. Uniqueness of the answers is Premise 1: the run is a function of $\mathcal{S}$, so no alternative trace is consistent with it. □

Corollary 5.3 (Causal attribution is total)**.** *For every value in the final store there is a finite chain of events, each naming a source site, terminating in declarations, that produced it.*

Proof. Follow last-writer edges backwards from the value. Each step is well defined by Theorem 5.2 and strictly decreases position in the linear event order, so the chain is finite; it terminates only at an event with empty read set, which is a declaration. □

Remark 5.4 (Interrogability is bought by exclusion). Theorem 5.2 holds because no rule of $\mathcal{K}$ consults the model. Were a single step to depend on an inference, that step's cause would be model-internal, Premise 1 would fail, and the chain of Theorem 5.3 would terminate at an event whose explanation is not in the source. The guarantee is therefore a consequence of keeping the model off the run path, not a property obtained despite doing so.

5.2 The limit: a trace is a store of local terms

Interrogability is not omniscience, and we state its boundary sharply because an implementer is otherwise likely to assume the trace answers more than it does.

Definition 5.5 (Two probing operators)**.** For a target $x^\star$ in a graph, $\mathsf{seek}(x^\star)$ is the set of items reachable from $x^\star$ by admissible propagation; $\mathsf{nec}(W, x^\star)$ is the set of $u \in W$ whose removal strictly changes the resolving power of W for $x^\star$.

Theorem 5.6 (The trace implements seek, not nec)**.** *Questions answerable from $\mathrm{tr}(\mathcal{S})$ by Theorem 5.2 are exactly forward-reachability questions, i.e.* seek*. The question "which items were load-bearing for this result" is a* nec *question and is not expressible as a projection of the trace, however finely the trace is instrumented.*

Proof. Each event records the items it read, so trace projections compute the transitive closure of the reads relation from a target, which is $\mathsf{seek}(x^\star)$ by Theorem 3.7. That nec is not so computable is the query-separation result of Sachikonye (2025a, §15): two graphs with identical edge sets — hence identical under every retrieval query, and in particular under every projection of a trace recording only reads — yield different admissibility verdicts, because the floor is a minimum over *every* item, including items lying on no path from the target. Adding numeric attributes to events does not close the gap: the obstruction is expressibility, not complexity. □

Corollary 5.7 (Correct probing is a composition)**.** *A determination must compute* nec ∘ seek*: reach forward to the candidate set, then ablate to find which members are load-bearing. Neither operator substitutes for the other.*

Proposition 5.8 (Ablation is tractable)**.** *A non-seed item is necessary exactly when it dominates some reachable item, so* nec *is computable in near-linear time by dominator-tree construction (Lengauer and Tarjan, 1979).*

Proof. Sachikonye (2025a, §13.5); the dominator characterisation and its near-linear algorithm are standard (Cooper et al., 2001; Lengauer and Tarjan, 1979). □

Remark 5.9 (Three implementation hazards, two of them silent). First, the resolving measure must *grow* with resolving power; a minimum cut has the wrong sign, and the reachable-slice measure is the correct one. Second, an item's contribution must not count its own disappearance — doing so collapses nec onto seek and reproduces exactly

the defect the distinction exists to prevent. Third, seeds are a genuine boundary case: a seed reaching only itself dominates nothing yet is necessary, so Theorem 5.8 is stated for non-seed items and seeds require separate treatment. The second and third produce no visible symptom, so they should be written as tests before the operator is implemented.

6 Replay, re-execution, and the record

Theorem 6.1 (Replay asymmetry)**.** *Let $\mathcal{S}$ be run twice, with the model observing and depositing each time. Then the two traces are identical,* $\mathrm{tr}(\mathcal{S}) = \mathrm{tr}(\mathcal{S})$*, while the pair states differ: M after the second run strictly exceeds M after the first, and the model's reading of the second run need not agree with its reading of the first.*

Proof. Trace identity is Premise 1: the run is a function of the source and the initial store, both unchanged. Record advance is Theorem 4.7: each run deposits, and deposits are committing acts, so M strictly increases. Disagreement of readings is then admissible because the second reading is taken of a catalogue that the first deposit altered; by Theorem 4.13 the model cannot be both adaptive and traceless, so its state before the second reading differs from its state before the first. □

Corollary 6.2 (Reproducibility relocates)**.** *"Same script, same answer" is true of the trace and false of the system's reading of it. Reproducibility attaches to the source, the trace, and the provenance record, never to the model's output (Cheney et al., 2009).*

Corollary 6.3 (Divergence is not a defect)**.** *Two readings of the same script that disagree constitute correct behaviour, not a fault to be reported: the first determination committed, altering the structure the second is taken of.*

Proof. Immediate from Theorem 6.1; the analogous result for determinations over a catalogue is Sachikonye (2025a, Cor. 12.6). □

Proposition 6.4 (Re-execution accretes; there is no retraction)**.** *Re-running a cell that deposits adds contacts and advances M. There is no operation on the pair that decreases M or removes a committed contact.*

Proof. M is non-decreasing by Theorem 4.2 and each deposit increments it. A removal of a committed contact would decrease Char's domain without a committing act, contradicting Theorem 4.3's requirement that Char be preserved along chains of committing acts. □

Corollary 6.5 (Undo is a forward act)**.** *A user-facing "reset" must be implemented as a compensating deposit that* increments *the record, never as a rewind.*

Remark 6.6 (Notebook semantics, corrected)*.* A conventional notebook overwrites: re-running a cell replaces its output and the previous result is gone. Theorem 6.4 forbids that here for depositing cells. Cells that only compute — no deposit — may be re-run freely, being mere rebinding in the store; the distinction is syntactic and can be shown in the interface.

7 Termination: closure, not confidence

An orchestration layer must decide when a determination has done enough. We record the criterion, because the obvious one is provably wrong.

Definition 7.1 (Reachable class set)**.** For a seed v_0, $\mathcal{C}(v_0)$ is the set of targets reachable by admissible propagation through probes invoked so far, partitioned by endpoint-indistinguishability.

Definition 7.2 (Closure)**.** A determination is *closed* when, for every available but uninvoked probe, extension by that probe adds no new class to $\mathcal{C}(v_0)$.

Theorem 7.3 (Closure is strictly stronger than a confidence threshold)**.** *There are process graphs and thresholds θ at which a propagation's terminal alignment satisfies θ while the determination is not closed.*

Proof. Sachikonye (2025a, Thm. 14.3). The witness is two internally dense clusters each joined to the medium by one probe: a propagation through one satisfies any $\theta \leq 1$ trivially, its terminal alignment to its own target being at the floor, while the uninvoked probe reaches a class the first does not. □

Theorem 7.4 (Convergent or contested closure)**.** *Over a finite probe registry every determination terminates in exactly one of: (i)* convergent closure*, $\mathcal{C}(v_0)$ a single class, reported by its representative;*

(ii) contested closure, $\mathcal{C}(v_0)$ *stable with more than one class, reported as* decline *together with the classes found.*

Proof. Sachikonye (2025a, Thm. 14.5): the registry is finite, so $\mathcal{C}(v_0)$ can grow only finitely often and closure is reached by exhaustion; the two cases partition the stabilised set. □

Corollary 7.5 (Decline is a first-class output)**.** *Contested closure is a correct termination carrying the divergent classes. It is not a failure to be retried, and the system could not adjudicate between the classes in any case.*

Remark 7.6 (Budget never stops a determination). Budget selects which probe to invoke next — by water-filling where returns diminish, by 0/1 knapsack where a probe is all-or-nothing (Dantzig, 1957; Kellerer et al., 2004; Sachikonye, 2025a, §13.4). It does not decide whether to stop. A determination that has not closed should descend further or decline; exhausting a budget licenses neither a verdict nor a fabrication.

8 The blueprint

We collect the commitments an implementation must satisfy, each with a checkable predicate.

Definition 8.1 (Blueprint invariants)**.** (B1) **Invariant.** A quantity conserved under relabelling of items.

(B2) **Record.** A never-resetting counter; "undo" increments it (Theorem 6.5).

(B3) **Search.** Determination by search, not lookup: no cache of prior determinations.

(B4) **Deposit.** Every propagation deposits, including those whose output is discarded.

(B5) **Accretive.** No teardown-then-rebuild; a resting cut exists at every intermediate stage.

(B6) **No verdict.** No success/failure vocabulary in the runtime.

Proposition 8.2 (The invariants are independent)**.** *No (Bi) follows from the conjunction of the others.*

Proof. Sachikonye (2025a, Prop. 17.2) exhibits, for each i, a system satisfying every invariant but the i-th. □

Remark 8.3 (Two with immediate teeth). (B3) forbids the obvious optimisation of memoising a repeated question: caching a determination is not a speedup but a violation, since by Theorem 6.1 the second determination is taken of a different structure. (B2) forbids a rewind-style session reset. Note that (B3) does not forbid rate-limiting *probes* — it forbids caching *determinations*; a dry-run-by-default policy on an external search tool changes which probes are invoked, never whether a result is recomputed.

9 The prototype

We describe a Python realisation of the deterministic core sufficient to exhibit every construction above. It is a reference implementation, not a performance claim.

9.1 Structure

The prototype comprises a lexer, a Pratt parser for the expression cascade, a big-step evaluator implementing Theorem 3.2, a static graph pass implementing Theorem 2.6, and a JSON emitter.

```
item threshold = 0.7
item readings = [0.4, 0.8, 0.9]

funxn above(xs, t):
    item hits = 0
    considering all x in xs:
        given x > t:
            hits = hits + 1
    return hits

proposition Signal:
    motion Strong("enough readings clear bar")
    given above(readings, threshold) >= 2:
        support Strong with_confidence(0.9)
```

9.2 Emitted record

Each cell execution emits a JSON object carrying: the stage index; the store delta; the trace as a list of events per Theorem 3.3; the stage graph Γ_k as items and weighted contacts; the deposit summary; and the record counter after deposit. The store delta and the trace together satisfy Theorem 5.2: every atomic question is a projection of the emitted JSON.

Remark 9.1 (What the prototype deliberately omits). No model is invoked. The prototype emits the record a model would read, which is the separable half; Theorem 4.11 says a complete system needs the other half, and the interface to it is δ alone. The nec operator of Theorem 5.7 is likewise not implemented: the prototype computes seek, and reports that it does, rather than presenting reachability as necessity.

10 Discussion and limitations

10.1 Relation to existing practice

Retrieval-augmented generation places a store on the read path of a model and treats the model's output as the answer. Two differences are structural rather than incidental. First, here the store is not consulted *by* the propagator; it is the propagator's own record, and the model reads it afterwards. Second, retrieval implements seek (Sachikonye, 2025a, §13.5), and by Theorem 5.6 no improvement in retrieval yields nec.

Incremental and self-adjusting computation (Acar et al., 2006) shares the prefix-stability property of Theorem 2.9 but permits invalidation and recomputation; Theorem 6.4 forbids the analogous move here, the record being non-decreasing. Provenance systems (Cheney et al., 2009) record derivation histories much as Theorem 3.3 does; the difference is Theorem 6.2, which locates reproducibility in the provenance rather than in the answer.

10.2 What the development assumes and does not supply

Three assumptions are load-bearing and none is proved here. Premise 1 is a property of the implemented fragment, not of Turbulance in general: the resolution forms admit world-touching resolvers, and an implementation that enables them leaves the deterministic fragment and forfeits Theorems 3.5 and 5.2. Theorem 2.5 inherits non-completability of the medium from Sachikonye (2025c). The concavity premise underlying water-filling (Theorem 7.6) is a claim about the environment probed, not about the runtime, and should be recorded per probe kind rather than assumed.

10.3 Honest limitations

The endpoint-indistinguishability predicate of Theorem 7.1 is not supplied at the level of user goals: we prove closure is the right criterion without saying, for a particular application, when two results count as the same class. That is a genuine gap, and it is the one place where an implementation must make a decision the theory does not force.

We prove nothing about the model. Theorem 4.11 shows a model is required for individuation; it does not show any particular model is adequate, and no result improves if the model does.

Finally, Theorem 5.6 is a limitation of this architecture stated in its own terms: a complete deterministic trace, however finely instrumented, answers what happened and not what mattered. We regard stating it as preferable to the alternative of allowing reachability to be read as necessity.

10.4 The picture in one statement

A script is a graph; compiling it is a propagation; the compiler is deterministic, which makes every step of the propagation nameable in the source and forbids the compiler from knowing whose propagation it was; a model reads the record, cannot help but be changed by reading it, and thereby supplies the individuation the deterministic half cannot.

11 Conclusion

Ndombolo separates two things usually fused: the propagation, which is deterministic and therefore interrogable, and the record, which advances and therefore individuates. The separation is not a design preference. A deterministic compiler cannot distinguish a run from a copy of a run (Theorem 4.4); an adapting observer cannot read without being changed (Theorem 4.13). Placing the model off the run path is what makes atomic interrogation of the trace possible (Theorems 5.2 and 5.4), and placing it on the record path is what makes identity available at all (Theorem 4.9). One language, executed cell-wise, with a model that never touches the run and never leaves it unchanged.

References

Umut A. Acar, Guy E. Blelloch, and Robert Harper. Adaptive functional programming. *ACM Transactions on Programming Languages and Systems*, 28(6):990–1034, 2006.

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1 (4):379–474, 2009.

Keith D. Cooper, Timothy J. Harvey, and Ken Kennedy. A simple, fast dominance algorithm. *Software Practice and Experience*, 2001.

George B. Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–277, 1957.

Hans Kellerer, Ulrich Pferschy, and David Pisinger. *Knapsack Problems*. Springer, Berlin, 2004.

Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems*, 1(1):121–141, 1979.

Gordon D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, 1981.

Kundai Farai Sachikonye. The directional model: Causal knowledge graphs and generative models as one catalogue. Manuscript, 2025a.

Kundai Farai Sachikonye. Turbulance: A formal specification of a domain grammar for semantic computation. Manuscript, 2025b.

Kundai Farai Sachikonye. The irreducible residue: Individuation by negation in bounded resolvable spaces. Manuscript, 2025c.

Kundai Farai Sachikonye. Semantic runtime specifications: Resolution, provenance, and scheduling. Manuscript, 2025d.

Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, Cambridge, MA, 1993.